

High PE Utilization CNN Accelerator with Channel Fusion Supporting Pattern-Compressed Sparse Neural Networks

Jingyu Wang, Songming Yu, Jinshan Yue, Zhe Yuan, Zhuqing Yuan,
Huazhong Yang, Xueqing Li, Yongpan Liu

Department of Electronic Engineering & BNRist, Tsinghua University, Beijing, China
ypliu@tsinghua.edu.cn

Abstract—Recently CNN-based methods have made remarkable progress in broad fields. Both network pruning algorithms and hardware accelerators have been introduced to accelerate CNN. However, existing pruning algorithms have not fully studied the pattern pruning method, and current index storage scheme of sparse CNN is not efficient. Furthermore, the performance of existing accelerators suffers from no-load PEs on sparse networks. This work proposes a software-hardware co-design to address these problems. The software includes an ADMM-based method which compresses the patterns of convolution kernels with acceptable accuracy loss, and a Huffman encoding method which reduces index storage overhead. The hardware is a fusion-enabled systolic architecture, which can reduce PEs' no-load rate and improve performance by supporting the channel fusion. On CIFAR-10, this work achieves 5.63x index storage reduction with 2-7 patterns among different layers with 0.87% top-1 accuracy loss. Compared with the state-of-art accelerator, this work achieves 1.54x-1.79x performance and 25%-34% reduction of no-load rate with reasonable area and power overheads.

Index Terms—sparse CNN accelerators, pruning algorithms, pattern compression, channel fusion, no-load rate reduction

I. INTRODUCTION

Convolutional Neural Networks (CNN) have become very popular and powerful in various fields recently [1]. It has been able to compare to or even surpass the accuracy of humans in some tasks, e.g., face recognition [2] and object detection [3]. However, the calculation complexity and storage overhead of CNN are much higher than traditional methods [4].

Pruning algorithms have been proposed to reduce the complexity of CNN by forcing weights which have less influence on outputs to zeros. There are two categories of pruning algorithms, unstructured pruning [5][6] and structured pruning [7][8]. Unstructured pruning methods achieve higher sparsity and accuracy, but they are inefficient for hardware acceleration because of randomly distributed zeros. On the contrary, structured pruning methods are more hardware-friendly due to structurally pruned weights, while the sparsity and accuracy are relatively restricted. However, neither previous unstructured nor structured pruning approaches explore the fine-grained intra-kernel pattern pruning method. Recently a pattern pruning method [9] is introduced to reduce the pattern numbers of kernels in CNN. However, the number of nonzero values in each pattern is fixed, which may influence the accuracy.

Existing application-specific integrated circuit (ASIC) accelerators' [10][11][12][13][14] index storage methods are inefficient for pattern-pruned networks because they do not leverage the limited pattern numbers. Furthermore, because

of the inconstant sparsity of pruned networks, the PEs suffer from the no-load problem, i.e., some PEs are idle while others are calculating. Compared with unstructured-pruned ones, pattern-pruned networks are less affected by the no-load problem because of more stable sparsity with limited patterns. However, this problem still exists and needs hardware solutions. Recently an accelerator [15] tackles the no-load problem by balancing input activations. However, it does not solve the no-load problem caused by sparse weights.

The main contributions of this paper are listed as follows.

- 1) A Huffman-encoded pattern compression (HEPC) approach to reduce pattern numbers of irregularly pruned networks, and further reduce index storage overhead.
- 2) A fusion-enabled systolic architecture (FESA) supporting the channel fusion, which can improve the performance of the accelerator by reducing PEs' no-load rate.
- 3) Experiment results on CIFAR-10 [16] show that HEPC can achieve 2-7 patterns among convolution layers and 5.63x index storage reduction with only 0.87% top-1 accuracy loss. FESA can achieve 1.54x-1.79x performance and 25%-34% no-load rate reduction with reasonable area and power overheads.

II. MOTIVATION EXAMPLES

We want to design a hardware architecture with high PE utilization which supports the pattern-compressed(pruned) neural networks. As shown in Fig. 1, there are three motivations for this work. Motivation 1 and 2 are corresponding to the pattern compression algorithm, and motivation 3 is for the hardware architecture with high PE utilization.

Motivation 1: it is observed that irregularly pruned networks have large redundancy in kernel patterns. We use an irregularly pruned VGG16 [17] network for illustration. As shown in Fig. 1(a), the pattern of a kernel is defined as a boolean mask which indicates if the weight in the same position is nonzero. In different layers, the pattern numbers range from 57 to 512 with a mean value of 459, which is close to the theoretical maximum $2^9=512$. Furthermore, the distribution of kernels shows an aggregation feature, which indicates much redundancy. *How can we attain a network with limited patterns while keeping reasonable accuracy and sparsity?*

Motivation 2: we find that the existing weight storage scheme for sparse networks is inefficient. Former accelerator [11] discards all zeros and only retains nonzero weights and their indexes to reduce storage cost. As shown in Fig. 1(b), this

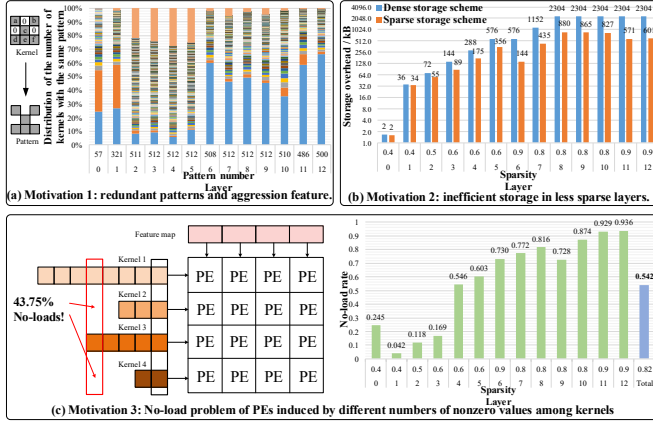


Fig. 1. Three motivation examples for this work illustrated by data of VGG16.

scheme is inefficient in convolution layers with less sparsity, e.g., layer 0 and 1. This is induced by the additional storage cost of indexes. If we manage to greatly reduce the pattern numbers of neural networks, *what storage scheme is more efficient for these pattern-compressed networks?*

Motivation 3: existing systolic array designs [12][13] suffer from the no-load problem. As shown in Fig.1(c), the systolic PE array receives four input weights from different channels in each clock cycle. Because the PE array is synchronous, the four input kernels will not be replaced until kernel 1 finishes calculation. So in latter clock cycles, the PE rows corresponding to kernel 2, 3 and 4 are no-load. In the VGG16 network, the no-load rate is very high in sparser layers, and the total no-load rate is 54.2%. *What hardware architecture is efficient to deal with the no-load problem?*

III. WORKFLOW OVERVIEW

For motivation 1 and 2, a Huffman-encoded pattern compression (HEPC) method is proposed to attain networks with limited patterns and reduced index storage overhead. For motivation 3, a fusion-enabled systolic architecture (FESA) is designed to achieve high PE utilization and performance.

The workflow of HEPC is shown in Fig. 2(a). First, an irregularly pruned model is compressed by an ADMM-based pattern compression method for motivation 1. Then the index storage of the pattern-compressed model is reduced by a Huffman-encoded weight storage method for motivation 2. Finally the tailored network is accelerated by FESA.

The workflow of FESA is shown in Fig. 2(b). First, the model is loaded to the main memory and transferred to the input buffer. Then the Huffman code table is programmed into the pattern decoder. A 16x16 systolic PE array is designed, which is dynamically linked to the PE memories by the memory linker row-wisely. Then the partial sums are added and stored in the output buffers. The controller is omitted for clarity. The details will be introduced part by part.

IV. HEPC: HUFFMAN-ENCODED PATTERN COMPRESSION

A. Background: ADMM Method for Neural Network Pruning

Alternating direction method of multipliers (ADMM) is a widely used optimization framework in the machine learning

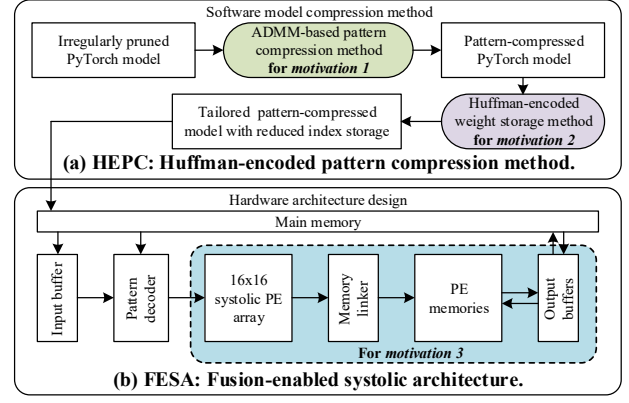


Fig. 2. The workflow of this work. HEPC is a software method intended for motivation 1 and 2, and FESA is a hardware architecture for motivation 3.

field [18]. Recently a study [6] introduces an ADMM-based method to irregularly prune CNN by introducing two auxiliary variable sets, $\{Z_i\}$ and $\{U_i\}$ (i represents the layer index), to add a differentiable loss which controls the sparsity. During training, it alternately updates these variables in one ADMM iteration to prune the network, shown as follows.

$$Z_i^{k+1} = \Pi_{S_i}(U_i^k + W_i^{k+1}) \quad (1)$$

$$U_i^{k+1} = U_i^k + W_i^{k+1} - Z_i^{k+1} \quad (2)$$

Here Π_{S_i} is the Euclidean projection onto the desired set $S_i = \{\text{card}(W_i) \leq l_i\}$, i.e., the number of nonzero weights in layer i should be less equal than l_i . It is proved that the analytical Euclidean projection onto S_i is to only keep l_i largest elements [18], so Z_i^{k+1} can be easily calculated. After updating Z_i^{k+1} and U_i^{k+1} , the additional gradients which controls the sparsity are added to update W_i by back propagation.

B. ADMM-Based Pattern Compression Method

Now we introduce the ADMM-based approach to compress the patterns of irregularly pruned models. First we enhance the target projection set S_i in equation (1) to support pattern compression, shown as follow.

$$S_i^P = \{W_i \mid \text{card}(W_i) \leq l_i, T(W_i^{oc,ic}) \in P_i\} \quad (3)$$

Here $W_i^{oc,ic}$ represents the kernel of the oc output channel and ic input channel of layer i , $T(\cdot)$ is the transformation function from a specific kernel to its binary pattern, and P_i represents the set containing target patterns of layer i .

The workflow of this method is shown in Fig. 3. First a probability density function (PDF)-aware approach is used to select the candidate patterns with the largest probability. Then we project the kernels to S_i^P by a distance-based pattern projection method. After the projection, the network is retrained for several epochs to regain accuracy followed by an irregular pruning to regain sparsity. This procedure will be repeated until the accuracy satisfies the expectation.

1) *PDF-Aware Pattern Selection:* In different convolution layers, the distributions of kernel patterns vary. Based on this observation, we compress the kernel patterns layer-wisely based on the PDF of the number of kernels with the same

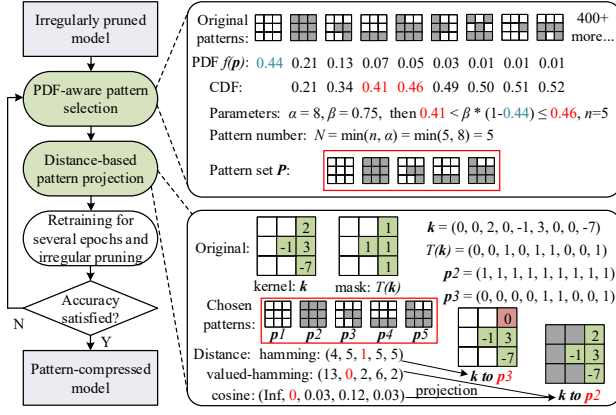


Fig. 3. The workflow of the ADMM-based pattern compression method.

pattern. There are two parameters to be decided, the pattern number N , and the target pattern set P . The footnotes of layer index are omitted for better display effect.

Denote the kernel size by $H \times W$ and the pattern of a kernel by $p \in \{0, 1\}^{H \times W}$, then the PDF of pattern p_m , denoted by $f(p_m)$, can be calculated by dividing the number of kernels with pattern p_m by total number of kernels. Without loss of generality, we suppose $f(p_m)$ is in descending order except for $f(p_1)$, where $p_1 = 0$ is the all-zero pattern.

$$f(p_m) \geq f(p_n), \forall 1 < m < n \quad (4)$$

Then the pattern number N and pattern set P are decided as follows. Here α, β are hyper parameters which control the maximum pattern number and threshold separately.

$$N = \min(n, \alpha), \quad (5)$$

$$s.t. \sum_{m=2}^{n-1} f(p_m) < \beta \cdot [1 - f(p_1)] \leq \sum_{m=2}^n f(p_m) \quad (6)$$

$$P = \{p_m \mid 1 \leq m \leq N\}$$

As shown in Fig. 3, we select the hyper parameters as $\alpha=8$ and $\beta=0.75$, then only $N=5$ patterns are needed to surpass the threshold. Finally p_1 and four other patterns with the largest PDF are selected to form the pattern set P .

In sum, the target pattern set P is obtained by selecting the all-zero pattern and the minimum number of nonzero patterns with top PDF, whose sum surpasses a threshold, while limiting the maximum number of patterns.

2) *Distance-Based Pattern Projection*: Denote the kernel by $k \in \mathcal{R}^{H \times W}$, and it needs to be mapped to P . We propose to map k to the pattern p with the minimum distance in P . So proper distance functions $D(k, p)$ are necessary. Using the aforementioned symbol $T(k)$ to denote the kernel-to-pattern transformation, we propose the following distance functions. Here $\otimes$ represents multiplying element-wisely.

$$D_h(k, p) = \|p - T(k)\|_1 \quad (7)$$

$$D_{\bar{h}}(k, p) = |p - T(k)| \cdot |k| \quad (8)$$

$$D_c(k, p) = 1 - \frac{k \cdot (k \otimes p)}{\|k\|_2 \cdot \|k \otimes p\|_2} \quad (9)$$

The hamming distance function D_h directly calculates the L1 norm of the residual of the target pattern and the kernel's

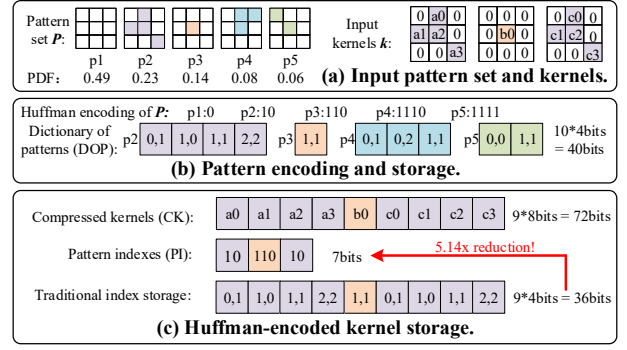


Fig. 4. A case study of the Huffman-encoded weight storage method. The index storage overhead is reduced by 5.14x.

pattern. The valued-hamming distance function $D_{\bar{h}}$ further considers the values in the kernel based on D_h . The cosine distance function D_c calculates the angle between the original kernel and the projected kernel. Using different distance functions may result in different patterns.

Once we find the pattern p^* with the minimum distance to k , the new kernel k^* is calculated by the element-wise multiplication of k and p^* , shown as below.

$$k^* = k \otimes p^* \quad (10)$$

As shown in Fig. 3, the original kernel k is projected to p_3 using D_h , p_2 using $D_{\bar{h}}$ and D_c for minimum distance.

In sum, we map kernels to the patterns with the minimum distances in P to reduce the number of patterns. Using different distance functions will affect the results of the mapping.

C. Huffman-Encoded Weight Storage Method

The pattern-compressed networks provide a new opportunity for weight storage reduction because the numbers of patterns are extremely limited, and the distribution of the patterns' frequency is very unbalanced.

As shown in Fig. 4(a), there are five patterns in P with descending PDF. We then show our method by a case study of the storage of three input kernels.

As shown in Fig. 4(b), we first encode the five input patterns using Huffman encoding technique, and store the dictionary of patterns (DOP) to record the indexes of the nonzero values in each pattern except for fixed p_1 . The storage cost of DOP is negligible because all kernels in one layer share the same DOP. The storage cost of DOP is calculated as follows.

$$S(\text{DOP}) = (\lceil \log_2(W) \rceil + \lceil \log_2(H) \rceil) \cdot \sum_{m=1}^N \|p_i\|_1 \quad (11)$$

The kernel's storage scheme is shown in Fig. 4(c). It consists of the compressed kernels (CK) and the pattern indexes (PI). CK are the concatenated nonzero weights, and PI are the concatenated pattern indexes of kernels. The indexes of weights are accessed from DOP according to PI. The index storage by PI is reduced by 5.14x compared with the traditional scheme, which stores all indexes for nonzero weights. The mean storage cost of PI for one kernel is calculated as follows.

$$S(\text{PI}) = - \sum_{m=1}^N f(p_m) \log_2 f(p_m) \quad (12)$$

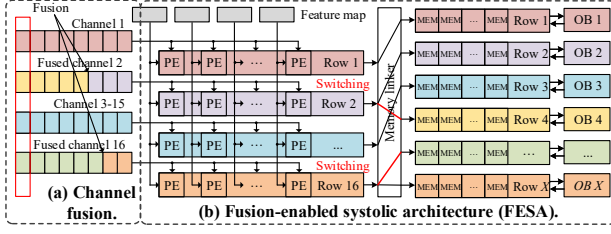


Fig. 5. The block design of the fusion-enabled systolic architecture (FESA).

In sum, we design the Huffman-encoded pattern compression method to compress the pattern number and reduce the index storage overhead.

V. FESA: FUSION-ENABLED SYSTOLIC ARCHITECTURE

A. No-Load Rate Reduction by Channel Fusion

As shown in Fig. 1(c), traditional systolic array architectures suffer from the no-load problem of PEs because of the irregular lengths of the compressed kernels. In pattern-compressed networks, this problem is weakened because the pattern numbers are greatly reduced and the possibility of the compressed kernels with the same length are higher. However, the no-load problem still exists and deteriorates performance. To reduce the no-load rate, we propose a channel fusion method, i.e., concatenating compressed kernels from different channels to one fused channel. The nonzero values' number of the fused channel is the sum of that of the source channels.

As shown in Fig. 5(a), in this calculation package, the 16 input channels consist of 14 ordinary channels (lines with the same color) and 2 fused channels (lines with different colors). In the fused channels, the original kernels' and the fused kernels' numbers of nonzero weights add up to the same length as other channels. By using the channel fusion technique, the no-load rate for this calculation package is reduced to zero. Furthermore, two additional kernels which are fused into the original kernels are calculated in the meantime, so the performance can be improved. To support the fused channel, a tailored hardware architecture is needed.

B. Fusion-Enabled Systolic Architecture

As shown in Fig. 5(b), the fusion-enabled systolic architecture mainly consists of four parts, the systolic PE array, the memory switcher, the PE memories and the output buffers. The 16×16 systolic array calculates multiple partial sums for different channels in one clock cycle, and each PE row is corresponding to one output channel. After one calculation of the PE array, the results are sent to the PE memories. Each PE row is linked to one PE memory row by the memory linker. The memory linker is designed as a finite state machine (FSM), which supports the mapping from PE rows to PE memory rows by comparing the output channel numbers of them. Each PE memory row contains the same number of PE memory blocks as the PEs in a PE row. During calculations, each PE memory block receives and accumulates the results from one PE. Each PE memory row is directly linked to one output buffer. After all partial sums are accumulated in a PE

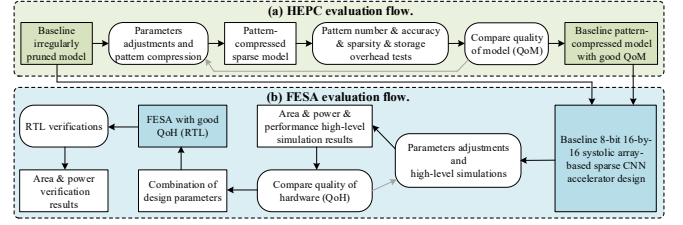


Fig. 6. The evaluation flow of HEPC(a) and FESA(b).

row, the summation of all partials sums in memory blocks is calculated and stored in the corresponding output buffer.

Because a fused channel contains multiple kernels of different weight channels, the PE memories and output buffers need to keep more channels of output partial sums than the physical input channel number of the systolic PE array. So more PE memory rows (output buffers) are needed to store the additional output channels' partial sums. As shown in Fig. 5(b), if the numbers of the PE memory rows and the output buffers are X , then there are $(X-16)$ additional PE memory rows and output buffers to store the partial sums of the additional fused channels. The second row of PEs is linked to the second PE memory row for the first three clock cycles, and then switched to the fourth PE memory row to calculate the results which belong to another output channel. The more PE memory rows are added, the more fused channels are supported. This is a trade-off between performance and area/power overheads, and we will give a design space exploration in the next section.

VI. EVALUATION

A. Methodology

We test HEPC based on PyTorch [19] framework, and test FESA based on both high-level behavioral simulations and register-transfer level (RTL) verifications. The evaluation flow is shown in Fig. 6. First the parameter space of HEPC is explored to achieve pattern-compressed models with appealing quality of model (QoM). These models and the original irregularly pruned models are the baseline models for hardware evaluation. Then we explore the design space of FESA to achieve appealing quality of hardware (QoH). Afterwards we implement FESA by RTL codes and the verification results are given. We will show the results part by part.

B. HEPC: Evaluation

The baseline networks for HEPC are irregularly pruned [6] VGG16 networks trained on CIFAR-10 and CIFAR-100 [16]. The network trained on CIFAR-10 is of 81.95% sparsity, 419 patterns on average, and 92.72% top-1 accuracy. The network trained on CIFAR-100 is of 81.95% sparsity, 450 patterns on average, and 72.72%/91.44% top-1/top-5 accuracy.

First, we explore the parameter space of HEPC method, which includes α , β , and the distance functions, while testing the sparsity, pattern number, accuracy and index storage reduction in the meantime. By comparing the results, we achieve networks with appealing QoMs, whose definition can be adjusted depending on the user-defined optimization target.

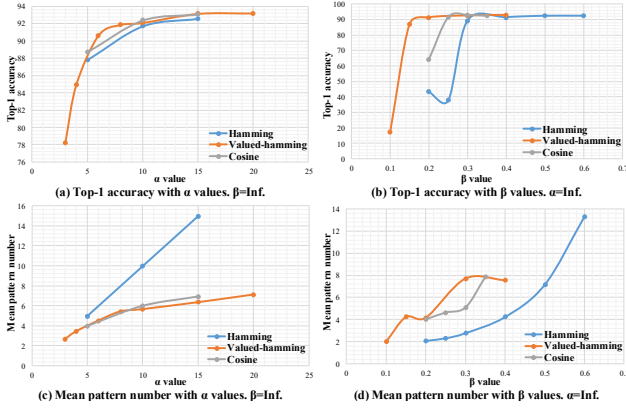


Fig. 7. The top-1 accuracy and mean pattern number with α and β of HEPC method. Results of different distance functions are tested on CIFAR-10.

The influence of different α , β , and distance functions on the top-1 accuracy of CIFAR-10 is shown in Fig. 7(a, b). The results on CIFAR-100 are similar. The top-1 accuracy is higher with larger α and β values because they allow more patterns, which may improve the perception efficacy of the network. The most effective distance function is the valued-hamming because it considers not only the existence of nonzero values, but also the amplitudes of them, which potentially maps the kernels to patterns more accurately.

The impacts of various α , β , and distance functions on the mean pattern numbers are shown in Fig. 7(c, d). The mean pattern numbers increase with larger α and β because the constraints are weaker. The mean pattern numbers of different distance functions do not have definite relationship in magnitude, but the incrementation is faster using the hamming distance because it neglects the amplitudes of nonzero values. This could lead to worse pattern projections and accuracy degradation, which needs additional patterns to recover. The results of valued-hamming and cosine distance functions are similar. So generally D_h and D_c are superior to D_h . However, the optimal results still need to be handmade.

Some results of competitive QoMs are shown in TABLE I, and we mark our choice of the baseline pattern-compressed networks for hardware in red font. On CIFAR-10, the chosen network achieves 2-7 patterns and 5.63x index storage reduction at the cost of 0.87% top-1 accuracy loss. Moreover, the sparsity is 0.53% higher. In the chosen network on CIFAR-100, we achieve 2-10 patterns and 3.47x index storage reduction at the cost of 0.35% top-1 accuracy loss and 1.38% sparsity loss. The top-5 accuracy is 0.45% higher.

C. FESA: Evaluation

First, we build a high-level simulator for the baseline 8-bit 16-by-16 systolic array-based sparse CNN accelerator design [12] in order to explore the design space of FESA and obtain the no-load rate, total clock cycles, area, and power of FESA with different design parameters. By comparing the results, we select parameters which can achieve appealing QoHs to implement the hardware. The QoH is decided by a user-defined trade-off between performance and overhead.

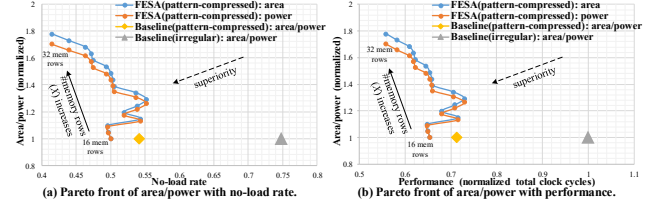


Fig. 8. Pareto fronts of area/power overheads with no-load rate and performance of HEPC. All results are tested on CIFAR-10.

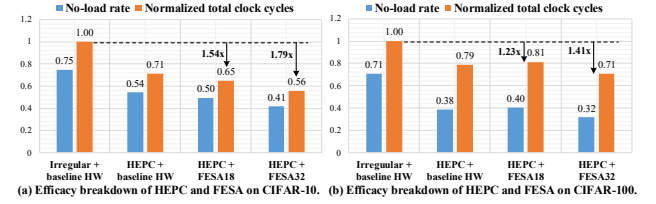


Fig. 9. Efficiency breakdown of HEPC and FESA on CIFAR-10/100.

The Pareto fronts of area (power) overhead and no-load rate (performance) of FESA are shown in Fig. 8 with superiority in left-bottom corners. Compared with the baseline which is the irregularly pruned network tested on the baseline hardware, FESA's no-load rate is reduced and its performance is improved to a large extent on CIFAR-10. The results on CIFAR-100 are similar. The improvements have two origins: HEPC is the software origin which contributes by compressing the pattern numbers, and FESA is the hardware origin which contributes by supporting the channel fusion.

As shown by the area/power curves of FESA, as the number of memory rows X increases, the area and power overheads increase, the no-load rate reduces and the performance increases. The no-load rate and performance are not monotonic to X due to the non-optimal mapping algorithm from the pattern-compressed networks to FESA. For RTL verification we select both $X=18$ and $X=32$ to implement FESA by HDL, namely FESA18 and FESA32.

The verification results of energy and power overheads are shown in TABLE II. Compared with the baseline architecture, FESA18 consumes 1.17x area and 1.73x power, while FESA32 consumes 1.89x area and 2.46x power. The incrementation of area is mainly due to the PE memories, and the incrementation of power is mainly caused by the memory switcher. The size of the PE memories is large because it is proportion to the number of memory rows X . The power consumption of the memory switcher is large because it synchronously switches the link between 16 PE rows and X PE memory rows without additional clock cycles to provide optimal performance.

The breakdowns of the reductions of the no-load rate and normalized total clock cycles on FESA18 and FESA32 are shown in Fig. 9. On CIFAR-10, by using the HEPC method to compress patterns, the no-load rate of the baseline hardware is reduced from 75% to 54%, and the normalized total clock cycles are reduced by 1.41x. Furthermore, the no-load rate is reduced to 50%, 41% and the normalized total clock cycles are reduced by 1.54, 1.79x on FESA18 and FESA32 respectively. The results are similar on CIFAR-100.

In sum, compared with the baseline architecture [12]

TABLE I
PATTERN-COMPRESSED AND HUFFMAN-ENCODED NETWORKS WITH COMPETITIVE QOMs. ALL THESE RESULTS ARE TESTED ON VGG16.

Dataset	Distance function	α	β	Sparsity	Pattern number avg / min / max	Top-1 accuracy	Top-5 accuracy	Index storage reduction
CIFAR-10	D_h	Inf	0.4	85.15% (+3.20%)	4.23 / 2 / 9	91.47% (-1.25%)	/	5.51x
	$D_{\bar{h}}$	10	0.3	82.48% (+0.53%)	3.85 / 2 / 7	91.85% (-0.87%)	/	5.63x
	D_c	15	0.3	80.67% (-1.28%)	4.62 / 2 / 13	92.88% (+0.16%)	/	6.27x
CIFAR-100	D_c	9	Inf	84.08% (+2.13%)	5.62 / 2 / 9	71.42% (-1.30%)	91.74% (+0.30%)	2.87x
	$D_{\bar{h}}$	16	0.5	82.01% (+0.06%)	5.84 / 2 / 12	71.86% (-0.86%)	91.62% (+0.18%)	3.54x
	D_c	10	Inf	80.57% (-1.38%)	5.69 / 2 / 10	72.37% (-0.35%)	91.89% (+0.45%)	3.47x

TABLE II
VERIFICATION RESULTS OF AREA AND POWER OVERHEADS.

Area / mm ²	PE	PE memory	Output buffer	Memory switcher	Total
Baseline	1.197	1.998	0.387	/	3.582
FESA18	1.188	2.248	0.233	0.514	4.184 (1.17x)
FESA32	1.190	3.996	0.411	1.164	6.761 (1.89x)
Power / mW					
Baseline	109.87	21.50	25.95	/	151.16
FESA18	128.74	16.34	28.42	97.82	271.33 (1.73x)
FESA32	128.88	29.09	50.25	178.92	387.14 (2.46x)

and supported by HEPC, FESA18 achieves 25%/31% no-load rate reduction and 1.54x/1.23x performance on CIFAR-10/CIFAR100 at the cost of 1.17x area and 1.73x power overheads. FESA32 achieves 34%/39% no-load rate reduction and 1.79x/1.41x performance on CIFAR-10/CIFAR100 at the cost of 1.89x area and 2.46x power overheads.

VII. CONCLUSION

CNN have become very powerful algorithms in various applications recently. This paper proposes a software-hardware co-design, which includes: 1) a Huffman-encoded pattern compression (HEPC) method to reduce kernel pattern numbers and the index storage overhead of sparse neural networks with little accuracy loss. 2) A fusion-enabled systolic architecture (FESA) which reduces the no-load rate and improves the performance of CNN accelerators by supporting the channel fusion. Experiment results show that HEPC can achieve 2-7 patterns among convolution layers and 5.63x index storage reduction with only 0.87% top-1 accuracy loss. FESA can achieve 1.54x-1.79x performance and 25%-34% no-load rate reduction with acceptable area and power overheads.

ACKNOWLEDGMENT

This work was supported in part by the National Key R&D Program under Grant #2019YFB2204204; in part by the National Natural Science Foundation of China under Grant #61674094, #61720106013, #61934005; and in part by the Beijing Innovation Center for Future Chip and BNRist.

REFERENCES

- [1] M. Z. Alom, T. M. Taha, C. Yakopcic, S. Westberg, P. Sidike, M. S. Nasrin, B. C. Van Esen, A. A. S. Awwal, and V. K. Asari, "The history began from alexnet: A comprehensive survey on deep learning approaches," *arXiv preprint arXiv:1803.01164*, 2018.
- [2] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "Arcface: Additive angular margin loss for deep face recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4690–4699.
- [3] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, "Object detection with deep learning: A review," *IEEE transactions on neural networks and learning systems*, 2019.

- [4] M. Shafique, T. Theodoridis, C.-S. Bouganis, M. A. Hanif, F. Khalid, R. Hafiz, and S. Rehman, "An overview of next-generation architectures for machine learning: Roadmap, opportunities and challenges in the iot era," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2018, pp. 827–832.
- [5] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," *arXiv preprint arXiv:1510.00149*, 2015.
- [6] T. Zhang, S. Ye, K. Zhang, J. Tang, W. Wen, M. Fardad, and Y. Wang, "A systematic dnn weight pruning framework using alternating direction method of multipliers," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 184–199.
- [7] S. Anwar, K. Hwang, and W. Sung, "Structured pruning of deep convolutional neural networks," *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, vol. 13, no. 3, p. 32, 2017.
- [8] C. Lemaire, A. Achkar, and P.-M. Jodoin, "Structured pruning of neural networks with budget-aware regularization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 9108–9116.
- [9] X. Ma, F.-M. Guo, W. Niu, X. Lin, J. Tang, K. Ma, B. Ren, and Y. Wang, "Pconv: The missing but desirable sparsity in dnn weight pruning for real-time execution on mobile devices," *arXiv preprint arXiv:1909.05073*, 2019.
- [10] A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, "Scnn: An accelerator for compressed-sparse convolutional neural networks," in *2017 ACM/IEEE 44th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2017, pp. 27–40.
- [11] Z. Yuan, J. Yue, H. Yang, Z. Wang, J. Li, Y. Yang, Q. Guo, X. Li, M.-F. Chang, H. Yang *et al.*, "Sticker: A 0.41-62.1 tops/w 8bit neural network processor with multi-sparsity compatible convolution arrays and online tuning acceleration for fully connected layers," in *2018 IEEE Symposium on VLSI Circuits*. IEEE, 2018, pp. 33–34.
- [12] J. Wang, Z. Yuan, R. Liu, H. Yang, and Y. Liu, "An n-way group association architecture and sparse data group association load balancing algorithm for sparse cnn accelerators," in *Proceedings of the 24th Asia and South Pacific Design Automation Conference*. ACM, 2019, pp. 329–334.
- [13] Z. Yuan, J. Wang, Y. Yang, J. Yue, Z. Wang, X. Feng, Y. Wang, X. Li, H. Yang, and Y. Liu, "A sparse-adaptive cnn processor with area/performance balanced n-way set-associate pe arrays assisted by a collision-aware scheduler," in *2019 IEEE Asian Solid-State Circuits Conference (A-SSCC)*. IEEE, 2019, pp. 61–64.
- [14] J. Song, Y. Cho, J.-S. Park, J.-W. Jang, S. Lee, J.-H. Song, J.-G. Lee, and I. Kang, "An 11.5 tops/w 1024-mac butterfly structure dual-core sparsity-aware neural processing unit in 8nm flagship mobile soc," in *2019 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2019, pp. 130–132.
- [15] J. Lee, J. Lee, D. Han, J. Lee, G. Park, and H.-J. Yoo, "Inpu: A 25.3 tflops/w sparse deep-neural-network learning processor with fine-grained mixed precision of fp8-fp16," in *2019 IEEE International Solid-State Circuits Conference (ISSCC)*. IEEE, 2019, pp. 142–144.
- [16] A. Krizhevsky, G. Hinton *et al.*, "Learning multiple layers of features from tiny images," Citeseer, Tech. Rep., 2009.
- [17] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.
- [18] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein *et al.*, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [19] A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer, "Automatic differentiation in pytorch," 2017.